

Stack-chan Procedural Character Runtime

Engineering Design Document

Prepared for Rob

July 1, 2026

- Verdict
- Engineering Design Document: Stack-chan Procedural Character Runtime
 - 1. Project goal
 - 2. What Victor teaches us
 - 3. Stack-chan hardware constraints
 - 4. Recommended software base
- 5. Proposed runtime architecture
 - 6. Core allocation
- 7. Data model
- 8. Inter-task communication
- 9. Runtime tasks
 - 9.1 MotionTask - 100 Hz
 - 9.2 FaceTask - 30-60 Hz
 - 9.3 IntentTask - 20-50 Hz
 - 9.4 SensorTask - 25-50 Hz
- 10. Motion design
 - 10.1 Spring model
 - 10.2 Saccade model
 - 10.3 Blink model
- 11. Servo adapter design
 - 11.1 Pitch control
 - 11.2 Yaw control
- 12. Procedural face renderer
- 13. Mood and behavior model
- 14. Corrected task skeleton
- 15. Proposed repo layout
- 16. Build strategy
- 17. MVP milestones
 - Milestone 1 - Hardware truth test
 - Milestone 2 - Non-blocking motion runtime
 - Milestone 3 - Procedural eyes
 - Milestone 4 - Mood layer
 - Milestone 5 - Character modes
- 18. Biggest risks
 - Risk 1 - Yaw servo assumption
 - Risk 2 - Blocking library calls
 - Risk 3 - Display stealing motion time
 - Risk 4 - Fake "AI personality" from canned states
- 19. Changes to make to the rough design immediately

- [20. First implementation target](#)
- [References](#)

Verdict

Your direction is right: **do not port Victor**. Use Victor as a reference for where Anki separated behavior, mood, animation, face rendering, and robot I/O, then build a much thinner runtime for Stack-chan.

The rough plan needs three major engineering corrections:

1. **Do not pin real-time actuation to Core 0 by default.** On ESP-IDF targets, Espressif documents Core 0 / PRO_CPU as typically handling Wi-Fi/Bluetooth protocol work, while Core 1 / APP_CPU is typically used for application tasks. For a Stack-chan that may use Wi-Fi, speech, OTA, or network AI calls, the safer default is: leave Core 0 less disturbed, run motion/render tasks on Core 1, and only move things after measurement. [1]
2. **Do not share `globalState` directly across cores.** Floats in a global struct are not a synchronization model. Use a single-slot FreeRTOS queue, mutex-protected snapshot, or seqlock-style double buffer. For this project, a **single-slot** `xQueueOverwrite()` **snapshot** is the cleanest.
3. **Stack-chan's yaw is not necessarily a normal positional hobby servo.** The M5Stack StackChan body is described as having a **360-degree continuous-rotation horizontal axis** and a **90-degree vertical axis**, with feedback servos. That means yaw may need a feedback/velocity controller, not a naive `Servo.write(90)` positional model. [6]

Also, there is a plain bug in the draft:

```
tiltServo.attach(constrain(current_tilt, 60.0f, 120.0f));
```

That should be a write, not an attach:

```
tiltServo.write(constrain(current_tilt, 60.0f, 120.0f));
```

`attach()` attaches a servo object to a pin. Calling it every frame with an angle value will break control.

Engineering Design Document: Stack-chan Procedural Character Runtime

1. Project goal

Build a lightweight C++ animation and personality runtime for Stack-chan that borrows the useful architectural ideas from Vector/Victor without inheriting Victor's heavyweight engine design.

The target system should:

- Generate **procedural facial animation** instead of relying only on canned assets.
- Run on **ESP32-S3 / CoreS3-class hardware**.
- Separate high-level intent from low-level physical motion.
- Keep face and servo motion alive even when AI, speech, or networking stalls.
- Support Stack-chan's existing Arduino C++ ecosystem first.
- Avoid the Moddable JavaScript path for this project.

The system should not attempt to clone Victor's full `vic-engine`. Victor's public repo is huge, archived, historical, and not a practical base for a small ESP32 robot. The `kercre123/victor` repo is archived read-only, and its README says the active/history-preserving work moved elsewhere while this repo is kept for history. [2]

2. What Victor teaches us

Victor's useful lesson is not "copy the code." The useful lesson is **the separation of concerns**.

In the public Victor tree, there is a clear split between `animProcess` and `engine`. The root contains both directories, and the `engine` directory includes behavior-heavy subsystems such as actions, AI components, animations, audio, block world, comms, components, mood system, robot interface, vision, and visualization. [2]

The animation process side contains `cozmoAnim`, with folders like animation, audio, backpack lights, face display, speech recognition, text-to-speech, and files like `animEngine.cpp`, `animComms.cpp`, and `cozmoAnimMain.cpp`. That is the "animation process / presentation process" side of the robot personality. [3]

The mood system is also worth studying conceptually. Victor's `emotion.h` describes an emotion as something that tracks a single emotion value, including value history, decay, add/set operations, and range limits. The mood manager then manages a selection of emotions for the robot and exposes methods such as triggering emotion events, adding to emotions, setting emotion values, getting emotion values, and computing simple moods. [4]

The bad fit for Stack-chan is the heavy central engine. Victor's `CozmoEngine::Update()` coordinates engine states such as stopped, loading data, connecting to robot, and running, while updating UI handlers, proto handlers, robot manager, web services, and other

subsystems. That makes sense for Victor's full robot stack, but it is too much gravity for an ESP32 face-and-servo runtime. [5]

Design takeaway

Use Victor's **conceptual layering**:

```
Mood / intent / behavior
↓
Animation target generation
↓
Face and motion runtime
↓
Hardware adapters
```

Do not use Victor's heavyweight central engine pattern.

3. Stack-chan hardware constraints

The M5Stack StackChan is based around CoreS3, which uses an ESP32-S3 dual-core 240 MHz processor with 16 MB flash and 8 MB PSRAM. The platform includes a 2-inch display, camera, proximity/ambient sensor, IMU, speaker, dual microphones, microSD, Wi-Fi, and BLE. [6]

The robot body adds the constraints that matter most for animation:

- Horizontal axis: 360-degree continuous rotation feedback servo.
- Vertical axis: 90-degree feedback servo.
- 12 RGB LEDs.
- Touch zones, IR, NFC, battery, and USB-C.
- M5Stack explicitly warns against forcing movable parts while motors are powered or controlled. [6]

This means the control runtime needs to distinguish between:

```
pitch: positional angle control
yaw: positional feedback control OR velocity-like continuous control
```

Treating both axes as plain hobby servos is probably wrong for the production design.

4. Recommended software base

Use `stack-chan/stackchan-arduino` **first**.

That repo is explicitly an Arduino Framework Stack-chan library. It supports config loading, servo control, and several servo types, including PWM SG90, Feetech SCS0009, and Dynamixel XL330. [7]

Its servo header defines `StackchanSERVO`, motion enums, axis enums, servo types, servo parameters, and methods such as `begin`, `moveX`, `moveY`, `moveXY`, `motion`, `turnX`, `getPosition`, and `isMoving`. [8]

The important catch: the existing servo API is designed around higher-level movement calls and canned motions. The implementation includes gesture-style methods like greet, laugh, nod, and refuse, and `turnX()` includes a blocking `vTaskDelay()` during movement. [9]

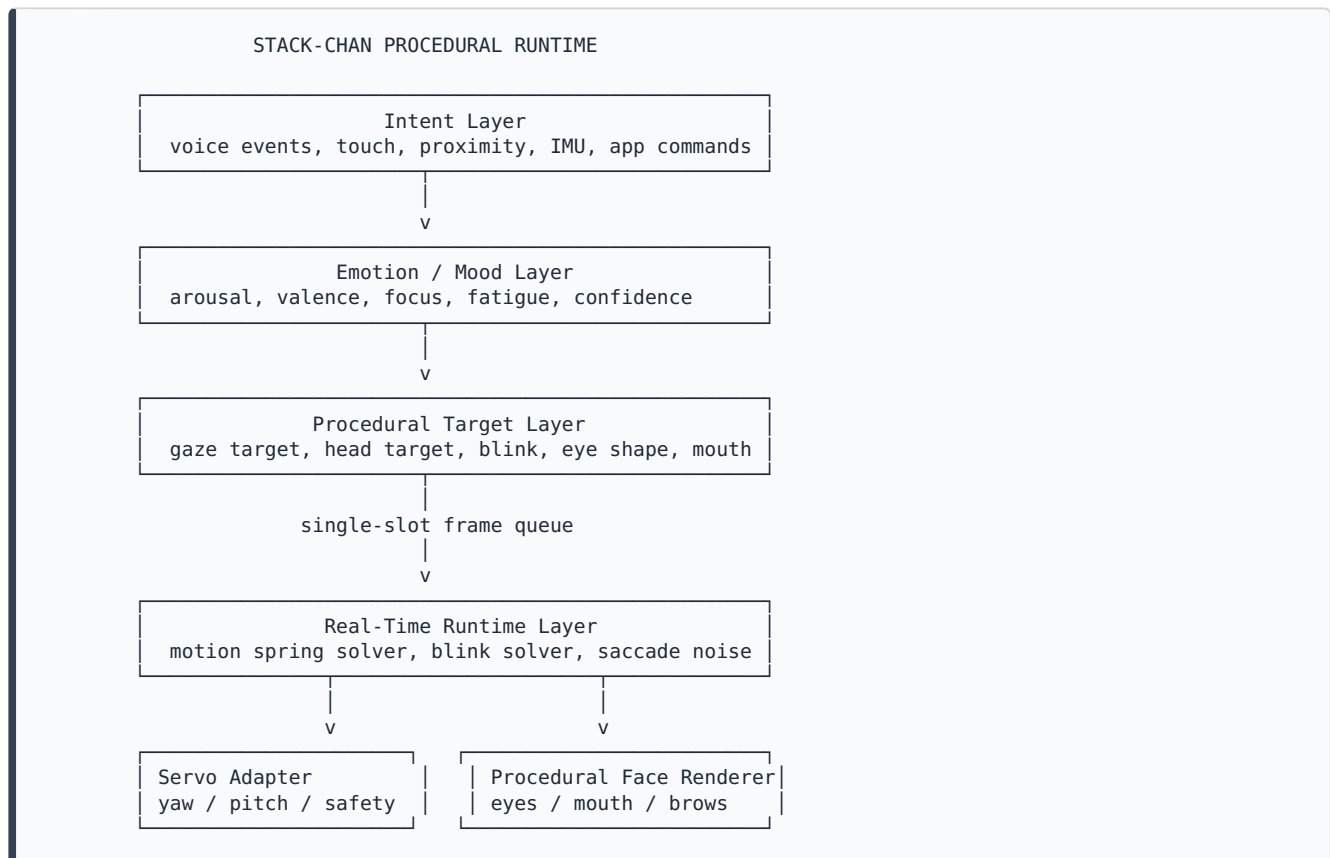
So the plan should be:

Use stackchan-arduino for board config, servo compatibility, and hardware bring-up.
Do not build the final procedural runtime around blocking canned motion calls.
Add or wrap a lower-level non-blocking servo adapter.

The alternative is the official M5Stack `StackChan` firmware repo. It has firmware, app, remote, and server directories, and the firmware uses ESP-IDF build tooling. That path is more “product firmware” oriented, but heavier for a quick C++ procedural animation project. [6]

5. Proposed runtime architecture

The original drawing has the right instinct, but I would revise it like this:



The key shift is this:

Intent does not command servos.
Intent publishes desired character state.
The runtime turns desired state into motion.

That keeps the character alive even if voice parsing, Wi-Fi, or cloud calls block.

6. Core allocation

The original split was:

```
Core 1: intent/state  
Core 0: actuation/render
```

I would not make that the default.

A safer initial mapping for ESP32-S3 Stack-chan is:

```
Core 0:  
  Wi-Fi / BLE / ESP-IDF system work  
  audio/network tasks if needed  
  lower-priority sensor polling if stable  
  
Core 1:  
  MotionTask      100 Hz  
  FaceTask        30-60 Hz  
  IntentTask      20-50 Hz  
  MoodTask        20-50 Hz
```

Espressif's FreeRTOS SMP documentation says ESP targets have Core 0 and Core 1, shared memory, task affinity, and `xTaskCreatePinnedToCore()` for pinning tasks. It also notes that Core 0 is typically responsible for Wi-Fi/Bluetooth protocol processing, while Core 1 is typically responsible for the rest of the application. [1]

So the first production rule should be:

```
Do not fight the radio stack until measurement says you need to.
```

Later, if you are running fully offline with no Wi-Fi/BLE/audio pressure, you can benchmark pinning motion to Core 0. But for a connected Stack-chan, I would start with motion/render on Core 1.

7. Data model

The `StateMatrix.h` sketch is a good start. I would expand it slightly and make it frame-based.

```

#pragma once
#include <Arduino.h>

struct EmotionalProfile {
    float arousal;    // 0.0 calm/sleepy, 1.0 startled/energetic
    float valence;    // -1.0 negative/withdrawn, 1.0 positive/social
    float focus;      // 0.0 wandering, 1.0 locked-on
    float fatigue;    // 0.0 fresh, 1.0 tired/sleepy
};

enum class YawMode : uint8_t {
    Angle,
    Velocity,
    Disabled
};

struct MotionTargets {
    YawMode yawMode;
    float yawDeg;    // desired yaw angle, if feedback position control exists
    float yawVel;    // desired yaw velocity, for continuous-rotation mode
    float pitchDeg;  // desired pitch angle
};

struct FaceTargets {
    float eyeOpen;    // 0.0 closed, 1.0 wide
    float squint;     // 0.0 round/neutral, 1.0 tight squint
    float eyeSmile;   // 0.0 neutral, 1.0 happy crescent
    float pupilX;     // -1.0 left, 1.0 right
    float pupilY;     // -1.0 down, 1.0 up
    float browTilt;   // -1.0 angry/skeptical, 1.0 curious/up
    float mouthSmile; // -1.0 frown, 1.0 smile
    float mouthOpen;  // 0.0 closed, 1.0 open/speaking
};

struct RobotFrame {
    uint32_t seq;
    uint32_t timestampMs;
    EmotionalProfile emotion;
    MotionTargets motion;
    FaceTargets face;
};

```

This frame becomes the contract between the “thinking” side and the “animation” side.

The runtime should keep the last valid frame and continue animating even when no new frame arrives.

8. Inter-task communication

Use a single-slot queue:

```
QueueHandle_t gFrameQueue;
```

Create it once:

```
gFrameQueue = xQueueCreate(1, sizeof(RobotFrame));
```

Publish from the intent/mood layer:

```
void publishFrame(const RobotFrame& frame) {
    xQueueOverwrite(gFrameQueue, &frame);
}
```

Read from the motion/render layer:

```
RobotFrame latest;

if (xQueueReceive(gFrameQueue, &latest, 0) == pdTRUE) {
    // Got newest target frame.
}
```

Why this is better than a global struct:

- No torn reads across cores.
- No stale partial emotional state.
- No lock contention in the 100 Hz motion loop.
- The queue always keeps the most recent target.
- Old frames are intentionally discarded.

This is important because the robot does not need every intermediate emotional thought. It needs the **latest desired character state**.

9. Runtime tasks

9.1 MotionTask - 100 Hz

Responsible for:

- Reading latest `RobotFrame` .
- Updating spring solvers.
- Applying saccades and idle body noise.
- Enforcing servo limits.
- Sending commands through the servo adapter.
- Detecting stale frames and fault states.

Target frequency:

```
100 Hz / 10 ms
```

9.2 FaceTask - 30-60 Hz

Responsible for:

- Procedural eye geometry.
- Blink state machine.

- Pupil movement.
- Mouth and speaking shapes.
- Brows / eyelids / expression overlays.
- Efficient screen flush.

Target frequency:

```
30 Hz minimum
60 Hz if display bandwidth allows
```

Do not force face rendering to run at 100 Hz unless measurement proves it is cheap enough. Display flushes can eat milliseconds. Servo motion should stay stable even if face rendering drops a frame.

9.3 IntentTask - 20-50 Hz

Responsible for:

- Reading high-level events.
- Updating mode state.
- Publishing `RobotFrame` .

Examples:

```
IDLE
ATTEND
LISTEN
THINK
SPEAK
REACT
SLEEP
ERROR
```

9.4 SensorTask - 25-50 Hz

Responsible for:

- Proximity.
- Touch.
- IMU.
- Optional face detection / camera events.
- Battery and thermal state.
- Publishing semantic events, not direct motor commands.

10. Motion design

10.1 Spring model

The spring model is the right approach. Keep it, but make the language precise.

The rough version used:

```
const float zeta = 0.85; // underdamped
```

That is not critically damped. Critical damping is:

```
zeta = 1.0f;
```

A value like 0.80-0.95 is intentionally slightly underdamped. That can look organic because it gives a tiny overshoot. So the production comment should be:

```
const float zeta = 0.88f; // intentionally slight overshoot; 1.0 = critical damping
```

Use this reusable spring:

```
struct Spring1D {
    float x = 0.0f;
    float v = 0.0f;
    float omega = 12.0f;
    float zeta = 0.88f;

    void reset(float value) {
        x = value;
        v = 0.0f;
    }

    float step(float target, float dt) {
        dt = constrain(dt, 0.001f, 0.040f);

        const float accel =
            omega * omega * (target - x)
            - 2.0f * zeta * omega * v;

        v += accel * dt;
        x += v * dt;
        return x;
    }
};
```

Clamping `dt` matters. If Wi-Fi, display, filesystem, or audio causes a scheduling hiccup, an unclamped spring step can jump violently.

10.2 Saccade model

The sine-based jitter is acceptable for a first test, but it will look synthetic after a few seconds because the pattern repeats.

Better model:

```
focus high    -> smaller gaze wander, longer fixation
focus low     -> larger gaze wander, shorter fixation
arousal high   -> faster saccades, wider eyes, shorter blink interval
valence high   -> softer eye smile, less defensive squint
valence low    -> more squint, lower brows, less approach motion
```

Use a fixation target that updates at random intervals:

```
struct SaccadeGenerator {
    float offsetX = 0.0f;
    float offsetY = 0.0f;
    uint32_t nextMs = 0;

    void update(uint32_t nowMs, const EmotionalProfile& e) {
        if (nowMs < nextMs) return;

        const float wander = (1.0f - e.focus) * 0.55f + e.arousal * 0.15f;

        offsetX = random(-100, 101) * 0.01f * wander;
        offsetY = random(-100, 101) * 0.01f * wander * 0.55f;

        const uint32_t minHold = e.arousal > 0.7f ? 120 : 280;
        const uint32_t maxHold = e.focus > 0.7f ? 1200 : 650;

        nextMs = nowMs + random(minHold, maxHold);
    }
};
```

Then use springs to ease the pupils/head toward that fixation. Do not instantly jump every frame.

10.3 Blink model

Blinking should be independent from high-level state. It should be a small procedural machine:

```
Open -> Closing -> Closed -> Opening -> Open
```

Suggested parameters:

```
Normal blink interval: 2-6 seconds
High arousal interval: 1-3 seconds
Tired interval: 0.8-2 seconds with slower reopening
Blink close time: 35-60 ms
Blink hold time: 20-50 ms
Blink open time: 50-100 ms
```

Blink should multiply against `face.eyeOpen` .

```
effectiveEyeOpen = targetEyeOpen * blinkOpenRatio;
```

This lets the character blink while happy, angry, sleepy, or surprised.

11. Servo adapter design

Do not let the intent layer call this:

```
servo.motion(Motion::greet);
servo.moveXY(..., 1000);
servo.turnX(...);
```

Those are useful for demos, but not for a non-blocking procedural runtime. The Stackchan Arduino implementation includes predefined motion sequences and blocking delays in places like `turnX()` . [9]

Create an adapter with a strict rule:

```
Only MotionTask writes to the servos.
All other code publishes targets.
```

Interface:

```
class IActuator {
public:
    virtual bool begin() = 0;
    virtual void writePitchDeg(float pitchDeg) = 0;
    virtual void writeYawAngleDeg(float yawDeg) = 0;
    virtual void writeYawVelocity(float yawVel) = 0;
    virtual void stop() = 0;
};
```

For first bring-up, you can wrap `StackchanSERVO` .

For production, you probably want lower-level immediate write methods, especially if you are using your own spring solver. The existing library already supports multiple servo types and exposes servo parameters like pin, start degree, offset, lower limit, upper limit, and move duration, which is useful for calibration. [8]

11.1 Pitch control

Pitch is simple:

```
float pitch = pitchSpring.step(targetPitch, dt);
pitch = constrain(pitch, pitchMin, pitchMax);
actuator.writePitchDeg(pitch);
```

11.2 Yaw control

Yaw depends on hardware mode.

Case A - yaw supports position feedback

Use angle control:

```
float yaw = yawSpring.step(targetYaw, dt);  
yaw = constrain(yaw, yawMin, yawMax);  
actuator.writeYawAngleDeg(yaw);
```

Case B - yaw is continuous rotation with readable position

Use a feedback controller:

```
error = wrapDegrees(targetYaw - measuredYaw);  
velocityCmd = kp * error - kd * measuredYawVelocity;  
velocityCmd = constrain(velocityCmd, -maxVel, maxVel);  
actuator.writeYawVelocity(velocityCmd);
```

Case C - yaw is continuous rotation without reliable feedback

Do not pretend it has absolute position.

For MVP:

```
Use pitch + eyes for most expression.  
Use yaw only for small timed sweeps.  
Recalibrate center on boot.  
Avoid absolute yaw claims.
```

This is the kind of hardware truth that prevents a “quick project” from becoming a week of servo weirdness.

12. Procedural face renderer

Stack-chan has a 320x240 display on CoreS3-class hardware. The face runtime should use simple geometric primitives, not heavy image animation. CoreS3's display is documented as 320x240, and the hardware has enough PSRAM for sprite buffering if used carefully. [10]

Recommended primitives:

Eyes:
rounded rectangles
superellipses
top/bottom eyelid curves
pupil circles/ovals
catchlight dots

Brows:
thick lines or rounded quadrilaterals

Mouth:
quadratic curve
filled oval for speaking
short line for neutral
curved smile/frown

Effects:
blush marks
sweat drop
sparkle
angry vein mark

Expression is generated from continuous parameters:

```
struct EyeGeometry {  
    float cx;  
    float cy;  
    float width;  
    float height;  
    float upperLid;  
    float lowerLid;  
    float pupilX;  
    float pupilY;  
    float squint;  
    float smile;  
};
```

The renderer should not know why the robot is happy, listening, or startled. It only receives geometry targets.

13. Mood and behavior model

Keep it thinner than Victor.

Victor has a real mood manager, emotion objects, event mapping, decay evaluators, scoring, simple mood types, and web/app stimulation hooks. That is conceptually useful, but too much for the first Stack-chan runtime. [11]

Use this first:

```
struct MoodState {  
    float arousal = 0.2f;  
    float valence = 0.4f;  
    float focus   = 0.6f;  
    float fatigue = 0.0f;  
};
```

Events modify mood:

```
enum class EventType {
    Boot,
    FaceDetected,
    UserNear,
    UserTouched,
    WakeWord,
    UserSpeaking,
    SpeechEnded,
    ThinkingStarted,
    ResponseStarted,
    ResponseEnded,
    IdleTimeout,
    Error
};
```

Example event effects:

```
FaceDetected:
    focus += 0.25
    valence += 0.10

WakeWord:
    arousal += 0.30
    focus += 0.35

ThinkingStarted:
    focus += 0.15
    arousal += 0.10
    mouthOpen = 0

ResponseStarted:
    valence += 0.10
    mouthOpen follows audio envelope

IdleTimeout:
    arousal -= 0.10
    focus -= 0.20
    fatigue += 0.05

Error:
    valence -= 0.35
    arousal += 0.20
    focus = 0.4
```

Then decay every frame:

```
mood.arousal = approach(mood.arousal, 0.20f, dt * 0.08f);
mood.valence = approach(mood.valence, 0.35f, dt * 0.04f);
mood.focus = approach(mood.focus, 0.55f, dt * 0.06f);
mood.fatigue = approach(mood.fatigue, 0.00f, dt * 0.02f);
```

This gives you Victor-style emotional continuity without Victor-style system weight.

14. Corrected task skeleton

This is the shape I would start with.

```

#include <Arduino.h>
#include "StateMatrix.hpp"

static QueueHandle_t gFrameQueue;

static RobotFrame makeNeutralFrame() {
    RobotFrame f {};
    f.seq = 0;
    f.timestampMs = millis();

    f.emotion = {
        .arousal = 0.20f,
        .valence = 0.45f,
        .focus = 0.75f,
        .fatigue = 0.0f
    };

    f.motion = {
        .yawMode = YawMode::Angle,
        .yawDeg = 0.0f,
        .yawVel = 0.0f,
        .pitchDeg = 0.0f
    };

    f.face = {
        .eyeOpen = 0.85f,
        .squint = 0.0f,
        .eyeSmile = 0.15f,
        .pupilX = 0.0f,
        .pupilY = 0.0f,
        .browTilt = 0.0f,
        .mouthSmile = 0.15f,
        .mouthOpen = 0.0f
    };

    return f;
}

void publishFrame(const RobotFrame& f) {
    xQueueOverwrite(gFrameQueue, &f);
}

```

Motion task:

```

void MotionTask(void* pv) {
    RobotFrame target = makeNeutralFrame();

    Spring1D pitch;
    Spring1D yaw;
    pitch.reset(0.0f);
    yaw.reset(0.0f);

    TickType_t wake = xTaskGetTickCount();
    uint32_t lastUs = micros();

    while (true) {
        RobotFrame incoming;
        if (xQueueReceive(gFrameQueue, &incoming, 0) == pdTRUE) {
            target = incoming;
        }

        const uint32_t nowUs = micros();
        float dt = (nowUs - lastUs) * 0.000001f;
        lastUs = nowUs;
        dt = constrain(dt, 0.001f, 0.040f);

        float pitchTarget = target.motion.pitchDeg;
        float yawTarget = target.motion.yawDeg;

        // Organic body motion.
        const float idleAmp =
            (1.0f - target.emotion.focus) * 3.5f
            + target.emotion.arousal * 1.0f;

        const float t = millis() * 0.001f;
        pitchTarget += sinf(t * 1.7f) * idleAmp * 0.20f;
        yawTarget += sinf(t * 1.1f) * idleAmp;

        const float pitchCmd = pitch.step(pitchTarget, dt);
        const float yawCmd = yaw.step(yawTarget, dt);

        // Hardware writes go through one adapter only.
        // actuator.writePitchDeg(pitchCmd);
        // actuator.writeYawAngleDeg(yawCmd);

        vTaskDelayUntil(&wake, pdMS_TO_TICKS(10));
    }
}

```

Intent task:

```

void IntentTask(void* pv) {
    RobotFrame frame = makeNeutralFrame();
    uint32_t nextDemoEvent = millis() + 3000;

    TickType_t wake = xTaskGetTickCount();

    while (true) {
        const uint32_t now = millis();

        // Demo event generator. Replace this with touch/proximity/voice events.
        if (now >= nextDemoEvent) {
            frame.seq++;
            frame.timestampMs = now;

            frame.emotion.arousal = 0.75f;
            frame.emotion.focus = 0.35f;
            frame.emotion.valence = 0.55f;

            frame.motion.yawDeg = random(-35, 36);
            frame.motion.pitchDeg = random(-12, 13);

            frame.face.eyeOpen = 1.0f;
            frame.face.squint = 0.0f;
            frame.face.mouthSmile = 0.2f;

            nextDemoEvent = now + random(2500, 6000);
        } else {
            // Gentle decay.
            frame.emotion.arousal =
                max(0.20f, frame.emotion.arousal - 0.002f);

            frame.emotion.focus =
                min(0.90f, frame.emotion.focus + 0.002f);
        }

        publishFrame(frame);

        vTaskDelayUntil(&wake, pdMS_TO_TICKS(20));
    }
}

```

Setup:

```

void setup() {
    Serial.begin(115200);

    gFrameQueue = xQueueCreate(1, sizeof(RobotFrame));

    RobotFrame neutral = makeNeutralFrame();
    publishFrame(neutral);

    xTaskCreatePinnedToCore(
        MotionTask,
        "MotionTask",
        4096,
        nullptr,
        3,
        nullptr,
        1
    );

    xTaskCreatePinnedToCore(
        IntentTask,
        "IntentTask",
        4096,
        nullptr,
        2,
        nullptr,
        1
    );
}

void loop() {
    vTaskDelay(pdMS_TO_TICKS(1000));
}

```

The important part is not the exact task placement yet. The important part is that **the motion task owns hardware writes** and **intent publishes snapshots**.

15. Proposed repo layout

```
stackchan-procedural-runtime/  
  platformio.ini  
  
  src/  
    main.cpp  
  
    persona/  
      StateMatrix.hpp  
      EmotionModel.hpp  
      EmotionModel.cpp  
      IntentEngine.hpp  
      IntentEngine.cpp  
      EventBus.hpp  
  
    motion/  
      Spring.hpp  
      Saccade.hpp  
      Blink.hpp  
      ActuationEngine.hpp  
      ActuationEngine.cpp  
      YawController.hpp  
      SafetyLimits.hpp  
  
    face/  
      ProceduralFace.hpp  
      ProceduralFace.cpp  
      EyeGeometry.hpp  
      MouthGeometry.hpp  
      ExpressionMapper.hpp  
      ExpressionMapper.cpp  
  
    io/  
      StackChanServoAdapter.hpp  
      StackChanServoAdapter.cpp  
      DisplayAdapter.hpp  
      DisplayAdapter.cpp  
      SensorAdapter.hpp  
      SensorAdapter.cpp  
  
    config/  
      RobotConfig.hpp  
  
    data/  
      expressions.yaml  
      calibration.yaml  
  
    test/  
      test_spring.cpp  
      test_emotion_decay.cpp  
      test_expression_mapping.cpp
```

The rule:

```
persona/ never touches hardware.  
motion/ never parses voice or intent.  
face/ never knows behavior states.  
io/ is the only place that knows the board/library details.
```

That separation is how you keep this from turning into another mini-monolith.

16. Build strategy

Start with PlatformIO and `stackchan-arduino` .

The repo README says to use it as a PlatformIO dependency, and its library manifest lists Arduino framework support for `espressif32` plus dependencies such as M5Unified, ESP32Servo, ServoEasing, ArduinoJson, Dynamixel2Arduino, YAMLDuino, and SCServo. [7]

The README and manifest appear slightly out of sync on versioning: the GitHub page shows a latest release `v0.0.6` , while `library.json` in the repo shows version `0.0.7` . [7]

For first bring-up, I would use the GitHub dependency directly:

```
[env:stackchan]
platform = espressif32
board = m5stack-cores3
framework = arduino
monitor_speed = 115200

lib_ldf_mode = deep

lib_deps =
  https://github.com/stack-chan/stackchan-arduino.git
```

Then pin to a release once the hardware build is stable.

17. MVP milestones

Milestone 1 - Hardware truth test

Goal: learn what the actual servo layer does on your specific Stack-chan.

Tasks:

- Build one official/basic Stack-chan Arduino example.
- Confirm display init.
- Confirm pitch limits.
- Confirm yaw behavior.
- Test whether yaw gives reliable position feedback.
- Log `getPosition()` if available for your servo type.
- Verify what calls block and for how long.

Exit criteria:

```
Pitch moves safely.
Yaw behavior is understood.
Display can draw a simple face.
No code forces parts against mechanical limits.
```

Milestone 2 - Non-blocking motion runtime

Goal: prove the queue/spring architecture.

Tasks:

- Add `RobotFrame` .
- Add single-slot FreeRTOS queue.
- Add `MotionTask` at 100 Hz.
- Add spring control for pitch.
- Add safe yaw mode.
- Add serial timing logs.

Exit criteria:

Motion loop runs at 100 Hz.
p95 loop jitter under ~5 ms.
No blocking Stackchan motion calls in the 100 Hz path.
Servo output continues while IntentTask pauses.

Milestone 3 - Procedural eyes

Goal: make the face feel alive before adding AI.

Tasks:

- Draw two procedural eyes.
- Add blink scheduler.
- Add pupil saccades.
- Add eye open/squint/smile mapping.
- Add 30-60 Hz face task.

Exit criteria:

Idle face breathes.
Blinking is non-blocking.
Gaze moves without servo motion.
Expression changes from frame parameters.

Milestone 4 - Mood layer

Goal: make behavior continuous instead of canned.

Tasks:

- Add `EmotionModel` .
- Add event input.
- Add mood decay.
- Add expression mapping.

Exit criteria:

Touch/proximity/voice-demo events change mood.
Mood decays smoothly.
Face and head respond without fixed animation clips.

Milestone 5 - Character modes

Goal: useful robot behavior.

Modes:

```
BOOT
IDLE
ATTEND
LISTEN
THINK
SPEAK
REACT
SLEEP
ERROR
```

Each mode should set target tendencies, not own physical execution.

Example:

```
LISTEN:
  focus high
  arousal medium
  mouth closed
  pupils track center/up
  head slightly tilted

THINK:
  focus medium
  arousal medium-low
  eyes glance away
  mouth neutral
  small head drift

SPEAK:
  valence follows response style
  mouthOpen follows audio envelope
  gaze alternates between user and thought offset
```

18. Biggest risks

Risk 1 - Yaw servo assumption

The draft assumes yaw is a positional servo. Stack-chan's body description says the horizontal axis is 360-degree continuous rotation with feedback. [6]

Mitigation:

```
Implement yaw as a strategy:  
PositionYawController  
VelocityYawController  
DisabledYawController
```

Do not hardcode one model.

Risk 2 - Blocking library calls

`stackchan-arduino` is useful, but its canned motion layer is not designed as a 100 Hz procedural motor runtime. Some methods include delays and gesture sequences. [9]

Mitigation:

```
Use library for bring-up/config.  
Add a raw/non-blocking adapter for runtime motion.  
Only one task writes to servos.
```

Risk 3 - Display stealing motion time

If display flush and servo control live in the same loop, animation may stutter.

Mitigation:

```
MotionTask: 100 Hz  
FaceTask: 30-60 Hz  
No full-screen redraw unless needed  
Use sprites or dirty rectangles  
Measure frame time
```

Risk 4 - Fake “AI personality” from canned states

A robot that just jumps between `happy`, `sad`, `thinking`, and `speaking` will feel cheap.

Mitigation:

```
Use continuous emotional parameters.  
Use procedural blink, saccade, and body drift.  
Let modes bias parameters instead of replacing motion.
```

19. Changes to make to the rough design immediately

Replace this:

```
Core 1: Intent & State  
Core 0: Physical Actuation  
Shared globalState struct  
FreeRTOS Queue (Non-Blocking IRQ)
```

With this:

```
Core 1:
  IntentTask
  MoodTask
  MotionTask
  FaceTask

Core 0:
  Wi-Fi / BLE / system / audio pressure where applicable

Communication:
  single-slot FreeRTOS queue of RobotFrame snapshots

Hardware rule:
  only MotionTask writes servos
  only FaceTask writes display
```

Replace this:

```
extern RobotSystemState globalState;
```

With this:

```
QueueHandle_t gFrameQueue;
RobotFrame latestFrame;
```

Replace periodic modulo triggers:

```
if (current_time % 5000 < 50)
```

With scheduled events:

```
if (current_time >= nextEventMs) {
  nextEventMs = current_time + random(2500, 6000);
}
```

Replace sine-only jitter with:

```
fixation targets + random hold time + spring easing
```

Replace “critically damped springs” with either:

```
zeta = 1.0f; // truly critically damped
```

or:

```
zeta = 0.85f; // intentionally underdamped for character
```

Do not call it both.

20. First implementation target

The quickest successful project is not “Vector on Stack-chan.”

The quickest successful project is:

```
A procedural idle/listen/speak face and head runtime
with emotional parameters,
non-blocking servo motion,
safe yaw handling,
and a clean adapter boundary.
```

That gives you a foundation you can grow into:

```
voice events
LLM response state
camera/proximity tracking
touch reactions
personality profiles
motion presets generated from curves
```

The core win is that Stack-chan should never freeze its face or head because some higher-level “brain” code got busy. That is the exact architectural bottleneck the rough plan is trying to avoid, and with the queue/snapshot plus procedural runtime split, you get there cleanly.

References

- [1] Espressif ESP-IDF FreeRTOS SMP documentation: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html
- [2] kercr123/victor repository: <https://github.com/kercr123/victor>
- [3] Victor animProcess source tree: <https://github.com/kercr123/victor/tree/snowboy/animProcess/src>
- [4] Victor mood system emotion header: <https://github.com/kercr123/victor/blob/snowboy/engine/moodSystem/emotion.h>
- [5] Victor CozmoEngine source: <https://github.com/kercr123/victor/blob/snowboy/engine/cozmoEngine.cpp>
- [6] M5Stack StackChan repository: <https://github.com/m5stack/StackChan>
- [7] stack-chan Arduino repository: <https://github.com/stack-chan/stackchan-arduino>
- [8] Stackchan servo header: https://github.com/stack-chan/stackchan-arduino/blob/main/src/Stackchan_servo.h
- [9] Stackchan servo implementation: https://github.com/stack-chan/stackchan-arduino/blob/main/src/Stackchan_servo.cpp
- [10] M5Stack CoreS3 documentation: <https://docs.m5stack.com/en/core/coreS3>

[11] Victor mood system directory: <https://github.com/kercre123/victor/tree/snowboy/engine/moodSystem>